

Reporte de Práctica

Autenticación utilizando OAuth 2.0

Módulo 5. Programación con PHP

Alumno: César Cedillo Rodríguez

Módulo: Bases Tecnológicas para el Servicio Público 2

Práctica: Autenticación utilizando OAuth 2.0

Introducción

En el desarrollo de aplicaciones web modernas, la seguridad representa uno de los aspectos más importantes, especialmente cuando se trabaja con APIs REST que exponen información sensible o permiten operaciones críticas sobre los datos. Garantizar que solo los usuarios autorizados puedan acceder a determinados recursos es fundamental para preservar la integridad, confidencialidad y disponibilidad de la información.

Para atender esta necesidad, existen estándares como OAuth 2.0, un protocolo ampliamente adoptado que permite la autorización segura mediante el uso de tokens, evitando el intercambio directo de credenciales entre aplicaciones. OAuth 2.0 resulta especialmente útil cuando se requiere integrar servicios de terceros o cuando una API debe ser consumida por múltiples clientes de forma controlada.

En esta práctica se implementó la autenticación OAuth 2.0 en una API REST desarrollada con Laravel, utilizando Keycloak como servidor de identidad. El objetivo fue proteger los endpoints privados de la API, validar el acceso mediante tokens JWT y comprobar su correcto funcionamiento a través de pruebas realizadas con Postman.

Objetivo de la práctica

Implementar un mecanismo de autenticación y autorización basado en OAuth 2.0 para asegurar una API REST en Laravel, utilizando Keycloak como proveedor de identidad, y verificar que los recursos protegidos solo puedan ser accedidos mediante un token de autorización válido.

Recursos utilizados

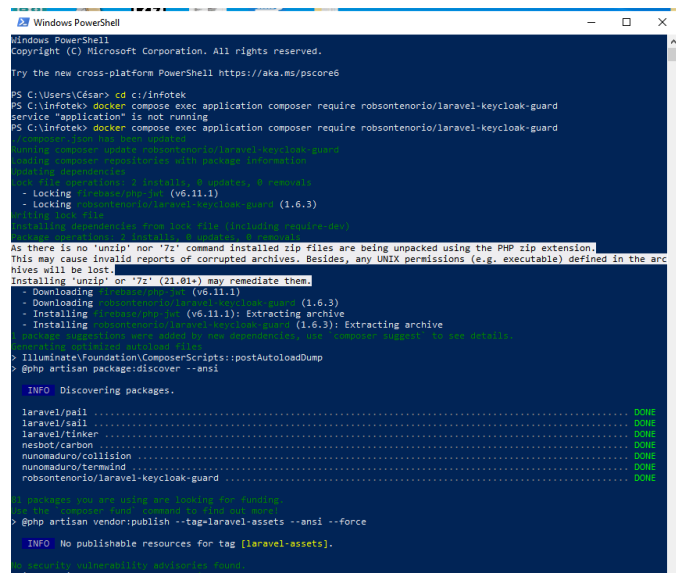
- Git
- Docker y Docker Compose
- Visual Studio Code
- Postman
- Laravel
- Keycloak
- Librería laravel-keycloak-guard

Desarrollo de la práctica

Etapa 1. Configuración de la autenticación

Para el desarrollo de esta práctica se retomó la API REST creada previamente en el módulo, realizando un respaldo del proyecto antes de aplicar cambios. Se inició el entorno de trabajo levantando los contenedores de Laravel y MariaDB mediante Docker, verificando el correcto funcionamiento de la aplicación desde el navegador en la URL <http://localhost:8000>.

Posteriormente, se instaló la librería `laravel-keycloak-guard`, la cual facilita la integración entre Laravel y Keycloak, permitiendo la validación automática de tokens JWT. Una vez instalada, se publicó su archivo de configuración y se añadieron las variables necesarias en el archivo `.env`, incluyendo la clave pública del realm, el identificador del cliente y las opciones relacionadas con la carga del usuario y el token decodificado.



```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Try the new cross-platform PowerShell https://aka.ms/pscore6

PS C:\Users\César> cd c:\infotek
PS C:\infotek> docker compose exec application composer require robsontenorio/laravel-keycloak-guard
service "application" is not running
PS C:\infotek> docker compose exec application composer require robsontenorio/laravel-keycloak-guard
[Truncated output showing package installation details]
Installing robsontenorio/laravel-keycloak-guard
- Downloading robsontenorio/laravel-keycloak-guard (v1.6.3)
- Installing robsontenorio/laravel-keycloak-guard (v1.6.3): Extracting archive
- Locking robsontenorio/laravel-keycloak-guard (v1.6.3)
- Writing lock file
- Updating dependencies to latest version (including require-dev)
[Truncated output showing package discovery and installation]
INFO: Discovering packages.
laravel/pail ..... DONE
laravel/sail ..... DONE
laravel/tinker ..... DONE
nesbot/carbon ..... DONE
nunomaduro/collision ..... DONE
nunomaduro/termwind ..... DONE
robsontenorio/laravel-keycloak-guard ..... DONE

[Truncated output showing the final state of the project]
INFO: No publishable resources for tag [laravel-assets].
[Truncated output showing the final state of the project]
```

También se modificó el archivo `auth.php` para establecer el guard api como predeterminado y configurar Laravel para utilizar el driver de Keycloak en las peticiones autenticadas.

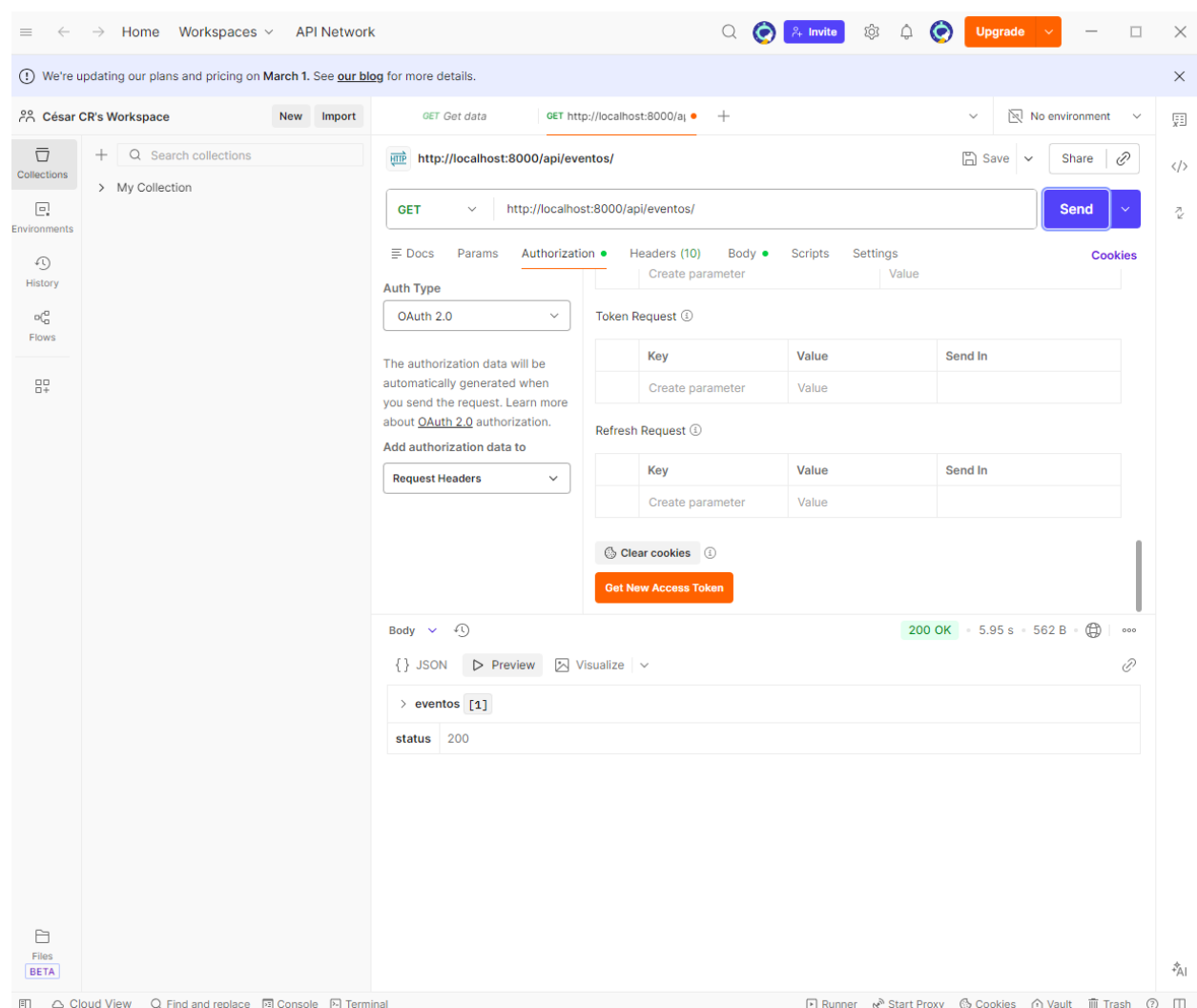
Protección de endpoints

Con la autenticación configurada, se procedió a proteger los endpoints de la API. Se definieron como rutas públicas aquellas que permiten consultar eventos y ponentes, mientras que las rutas que permiten crear, actualizar o eliminar registros, así como todas las operaciones relacionadas con asistentes, se agruparon bajo el middleware `auth:api`.

Esta configuración garantiza que Laravel valide la autenticación antes de permitir el acceso a los recursos protegidos, aplicando correctamente el control de acceso basado en tokens.

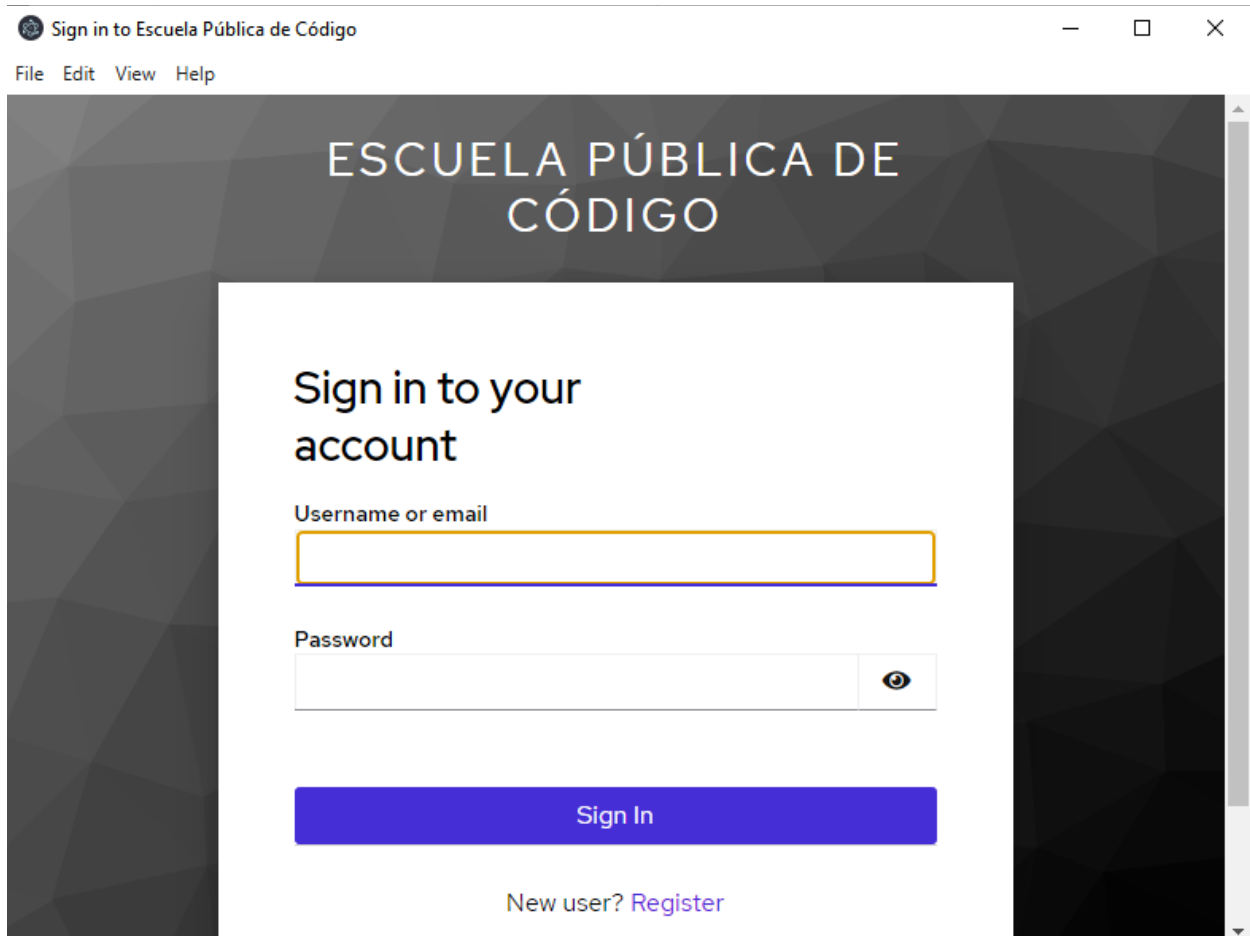
Etapa 2. Pruebas de la API REST

Las pruebas se realizaron utilizando Postman. En primer lugar, se verificó el acceso a los endpoints públicos, obteniendo respuestas correctas con código de estado 200, lo que confirmó la disponibilidad de la API.



Posteriormente, se intentó acceder a un endpoint privado sin proporcionar autenticación, obteniendo una respuesta con código 401 y el mensaje "Unauthenticated", lo que demostró que el middleware estaba funcionando correctamente.

Para validar el acceso autorizado, se configuró OAuth 2.0 en Postman utilizando el flujo Authorization Code, obteniendo un token JWT desde el servidor de identidad Keycloak. Una vez aplicado el token a la petición, se volvió a invocar el endpoint privado, obteniendo una respuesta exitosa con código 200 y el contenido esperado en formato JSON.



The image shows a web browser window with the title "Sign in to Escuela Pública de Código". The browser's menu bar includes "File", "Edit", "View", and "Help". The page has a dark, geometric background. At the top, the text "ESCUELA PÚBLICA DE CÓDIGO" is displayed in a light, sans-serif font. Below this, a white rectangular box contains the login form. The form starts with the heading "Sign in to your account". It then has two input fields: "Username or email" and "Password". The "Username or email" field is a simple white box with a yellow border. The "Password" field is a white box with a toggle icon (an eye) to its right. Below these fields is a blue "Sign In" button. At the bottom of the form, there is a link that says "New user? Register".

Sign in to Escuela Pública de Código

File Edit View Help

ESCUELA PÚBLICA DE CÓDIGO

Sign in to your account

Username or email

Password

New user? [Register](#)

ESCUELA PÚBLICA DE CÓDIGO

Register

* Required fields

Username *

CesarCR

Value contains invalid character.

Password *

.....



Confirm password *

.....



Email *

crciq2010@gmail.com

First name *

Cesar

Last name *

Cedillo

Register

[« Back to Login](#)

Resultados obtenidos

Se logró integrar exitosamente OAuth 2.0 en la API REST desarrollada con Laravel.

Los endpoints públicos y privados quedaron correctamente diferenciados.

Las peticiones sin token fueron rechazadas adecuadamente.

Las peticiones autenticadas con un token válido fueron procesadas de manera satisfactoria.

Se comprobó la correcta interacción entre Laravel, Keycloak y Postman.

Conclusiones

La implementación de OAuth 2.0 permitió fortalecer significativamente la seguridad de la API REST, evitando accesos no autorizados y delegando la autenticación a un servidor de identidad especializado. El uso de Keycloak simplificó la gestión de usuarios y tokens, mientras que la librería laravel-keycloak-guard facilitó la integración con Laravel.

Esta práctica demuestra la importancia de aplicar estándares de seguridad en el desarrollo de servicios web modernos y resalta el valor de utilizar herramientas y protocolos ampliamente aceptados en el entorno profesional.

Reflexión

Durante el ejercicio de reforzamiento se intentó invocar nuevamente endpoints privados de la API, como la creación y actualización de eventos y ponentes, los cuales anteriormente devolvían un error de autenticación al no contar con credenciales válidas. En esta ocasión, al incluir correctamente el token de autorización JWT obtenido desde Keycloak en el encabezado de la petición, Laravel respondió de manera satisfactoria, permitiendo el acceso a los recursos protegidos.

Esta experiencia permitió comprender de forma práctica el funcionamiento de OAuth 2.0 como un mecanismo que separa la autenticación del acceso a los recursos, evitando el uso directo de credenciales. Tal como se menciona en la literatura especializada, OAuth 2.0 utiliza tokens seguros y temporales que reducen los riesgos de exposición de información sensible y facilitan la integración entre aplicaciones.

Asimismo, resulta evidente que este protocolo ya forma parte de la vida cotidiana de los usuarios, por ejemplo, al iniciar sesión mediante cuentas de Google, Facebook o Microsoft. Implementar OAuth 2.0 en desarrollos propios no solo mejora la seguridad, sino que también brinda una experiencia de usuario más profesional y confiable, abriendo la puerta a integraciones más avanzadas y a soluciones alineadas con estándares utilizados en entornos reales.

Token

MANAGE ACCESS TOKENS

All TokensDelete

Práctica 2

Token DetailsUse Token

Token Name

Práctica 2

Access Token

eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUiIiwia2kiOiA6ICJYNHJsdjloLUlRcG1hLU00aEx2RzZS

Token Details

Use Token

Token Name

Práctica 2

Access Token

eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUiIiwia2kiOiA6ICJYNHJsdjloLUlRcG1hLU00aEx2RzZS

Token Type

Bearer

expires_in

3600

refresh_expires_in

1800

refresh_token

eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUiIiwia2kiOiA6ICJYNHJsdjloLUlRcG1hLU00aEx2RzZS

eyJhbGciOiJSUzI1NiIsInR5cCIgOiAiSldUiIiwia2kiOiA6ICJYNHJsdjloLUlRcG1hLU00aEx2RzZS

not-before-policy
0
session_state
a7d79970-c763-40fb-9ca0-2ba87df74eed
scope
openid profile email
access_token_url
https://idm.epc.gob.mx/realms/escuela-publica-de-codigo/protocol/openid-connect/token
client_id
infotec-api
client_secret
grant_type
authorization_code
password
username
client_authentication
header
timestamp
1770403389835